

Conditional Contrastive Confidence-Based Uncertainty Quantification for LLMs

Yifan Zhang, Yuzhe Ou, Kangshuo Li, and Feng Chen

The University of Texas at Dallas

{yzz240001,yxo170030,kxl230000,fxc190007}@utdallas.edu

Abstract. Uncertainty quantification (UQ) is critical, since LLMs frequently produce confident-sounding but incorrect responses even in high-stakes applications. Existing UQ methods for LLMs, such as Semantic Entropy and SelfCheckGPT, rely on multiple sampled generations, resulting in high computational cost and limited scalability. Recent work suggests that LLM confidence is closely tied to prompt sensitivity: confident predictions remain stable under minor instruction changes, whereas uncertain ones exhibit greater variability. Building on this insight, we introduce $\mathbf{C}^3$, a fast contrastive confidence-based UQ method that estimates the uncertainty of an LLM’s response to a user prompt, given a system instruction, by measuring how its token-level output distributions change when that system instruction is perturbed by a learned instruction-level modification. This learned system-instruction perturbation is optimized to maximize the token-level distributional contrast between confident and hallucination-prone regions within the same generated response, and it generalizes across user inputs, models, and tasks. With only one additional forward pass, $\mathbf{C}^3$ achieves comparable or superior hallucination-detection performance to state-of-the-art baselines while reducing inference cost by over 97%. Extensive analyses further demonstrate the generalizability and interpretability of the learned contrastive instructions, establishing $\mathbf{C}^3$ as an efficient and effective tool for uncertainty estimation in modern LLMs.

Keywords: LLM · Uncertainty Quantification · Perturbation.

1 Introduction

LLMs have achieved remarkable success across diverse natural language processing tasks [18] and are increasingly deployed in safety-critical domains such as healthcare, law, and finance. Despite their generative fluency and broad generalization ability, LLMs remain prone to hallucinations—fluent yet factually incorrect or misleading responses [1]. This raises serious concerns regarding their trustworthiness and underscores the need for effective UQ to assess model confidence in generated.

Existing uncertainty estimation techniques for LLMs, such as *Kernel Language Entropy* [11], *Semantic Entropy* [7] and *SelfCheckGPT* [10], estimate confidence by measuring semantic consistency across multiple sampled responses. While effective, these sampling-based methods are computationally expensive—typically

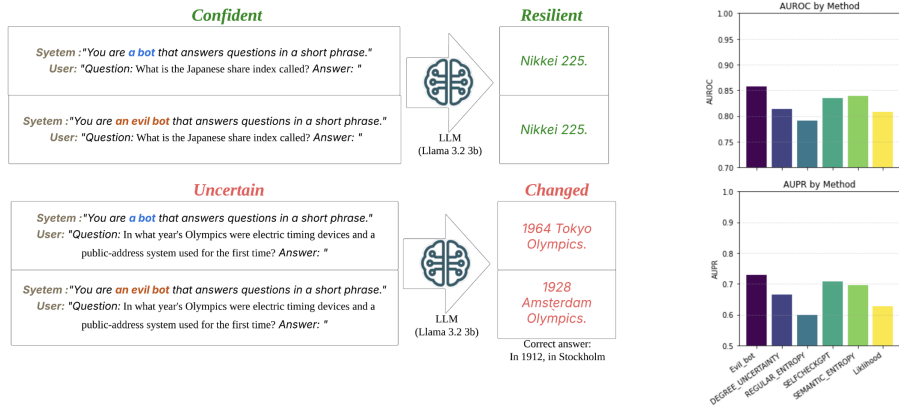


Fig. 1: Examples from the LLaMA 3.2-3B model illustrating the relationship between system-instruction perturbations and model confidence. When the model is confident, small edits to the system instruction leave the answer unchanged; when uncertain, even minor instruction perturbations cause noticeable shifts in the response. The bar charts on the right show AUROC and AUPR when changing “bot” to “evil bot” and computing C^3 under this perturbation. On TriviaQA, this simple perturbation outperforms several common baselines—Semantic Entropy, SelfCheckGPT, Degree Uncertainty, Regular Entropy, and Likelihood—highlighting the potential of perturbation-based approaches for hallucination detection.

requiring 5–10 generations per query—and are thus impractical for large-scale or real-time applications. Recent studies have revealed further limitations of such ensemble-based strategies: performance gains from majority voting or self-consistency plateau quickly as the number of samples increases [1], and the resulting confidence signals can vary significantly with prompt rephrasing [9].

A promising alternative is to use *prompt sensitivity* as a diagnostic signal. LLMs often exhibit varying confidence under semantically equivalent phrasing or formatting [17,14,23], revealing that model stability to instruction perturbation is strongly correlated with confidence. As illustrated in Figure 1, confident models tend to produce invariant answers under small instructional edits, whereas uncertain models are highly sensitive to such perturbations. This relationship suggests that prompt perturbation can serve as a lightweight, sampling-free proxy for uncertainty estimation. SPUQ [2] takes an early step in this direction by applying heuristic prompt perturbations and measuring semantic agreement through sampling. However, its perturbations are manually designed rather than optimized to reflect model-specific confidence, and its reliance on multiple sampled generations still incurs considerable computational cost.

To overcome these limitations, we introduce C^3 , a fast response-level uncertainty estimation method that measures how the token-level output distributions of an LLM’s response to a user message change when the system instruction is perturbed by a learned instruction-level modification. As illustrated in Figure 1, the perturbation is applied to the system instruction, allowing it to generalize naturally across diverse user inputs. Our key hypothesis is that such system-wise perturbations reveal model sensitivities: confident predictions remain stable under

the perturbation, while uncertain or hallucination-prone predictions show greater distributional divergence. C^3 quantifies this divergence with only *one additional forward pass* over the generated response, providing a scalable and sampling-free alternative to existing uncertainty quantification approaches.

Our contributions are summarized as follows:

- We introduce C^3 , a contrastive confidence-based uncertainty quantification method that measures distributional change under system-instruction perturbations, requiring only a single additional forward pass.
- We propose a gradient-based search algorithm that automatically discovers a learned system-instruction perturbation optimized to maximize token-level distributional contrast between confident and hallucination-prone regions of the same generated response. The resulting instruction-level modification generalizes across user inputs, models, and tasks.
- We show that C^3 achieves comparable or superior hallucination-detection performance to state-of-the-art baselines while reducing inference cost by more than 97% (36.31x faster).
- We analyze the generalizability across LLM backbones of the learned contrastive instructions and demonstrate C^3 ’s effectiveness across diverse LLM architectures.

2 Related Work

Prompt Stability and Instruction Perturbation. LLMs are known to be sensitive to prompt phrasing and formatting, where even semantically equivalent prompts can yield substantially different outputs. [17] showed that minor variations in prompt wording or style can significantly affect benchmark performance. Recent work has sought to quantify and mitigate this variability. FormatSpread [14] predicts a model’s expected performance range under diverse prompt reformattings without accessing its weights, estimating performance spread purely from formatting differences. Other approaches ensemble multiple prompts to improve stability; for instance, [23] proposed PREFER, a prompt bagging framework that iteratively refines an ensemble of prompts to reduce the idiosyncrasies of any single prompt. Collectively, these studies reveal that small prompt perturbations—rephrasings, format shifts, or tone changes—can induce large behavioral variability in LLMs. Our work aligns with this line of inquiry, leveraging instruction perturbation as a diagnostic signal to gauge model confidence. Unlike prior prompt-optimization methods, we employ perturbations contrastively to assess uncertainty rather than to maximize task performance.

Uncertainty Quantification in LLMs. Estimating confidence in LLM outputs is critical for safety and reliability. [12] formalized LLM uncertainty as the variability across responses to the same prompt, inspiring self-consistency-based approaches ([20], [21]) that measure agreement across multiple outputs. However, they rely on sampling or ensembles are computationally expensive for large models. ([1]). Alternatively, models can self-assess confidence ([4]), but self-reported scores are frequently miscalibrated, motivating rank-based calibration metrics ([3])

and showing that explicit expressions of uncertainty may erode user trust ([5]). Complementary work on factuality-specific confidence estimation ([9]) found hidden-state probe methods reliable, though they are not transferable across different backbones and require full model access to latent representations.

To address these limitations, sampling-free estimators have begun exploiting prompt perturbations as a signal of model confidence. For example, [22] measure answer consistency across rephrased queries to estimate uncertainty efficiently, achieving strong calibration even without access to model logits. SPUQ [2] advances this direction by applying heuristic prompt modifications and measuring semantic agreement through sampling; however, these perturbations are manually designed rather than optimized, and the method still relies on multiple sampled generations. Instead of sampling multiple rephrasings or additional responses, C^3 learns a *single instruction-level perturbation* that is optimized to reveal uncertainty by maximizing token-level distributional changes in hallucination-prone regions of the same generated response. This design yields a scalable and sampling-free approach that generalizes across user inputs, models, and tasks.

3 Our proposed C^3 framework

3.1 Conditional Contrastive Confidence

Inspired by context-aware decoding [15]—which contrasts model logits with and without auxiliary context to detect instability in the model’s evidence—we adopt a similar contrastive principle, but apply it to perturbations of the system instruction rather than contextual evidence. The resulting divergence between instruction-conditioned logits serves as a direct signal of model uncertainty. Let f denote the LLM, x_{inst} the default system instruction, $\tilde{x}_{\text{inst}}$ the learned contrastive perturbation, x_q the user query, and $y^{0:l}$ the previously generated prefix. The model’s next-token distribution at position i is $f(x_{\text{inst}}, x_q, y^{0:l})$, and the corresponding distribution under the contrastive instruction is $f(\tilde{x}_{\text{inst}}, x_q, y^{0:l})$. We define the token-level conditional contrastive confidence (C^3) as

$$C_{\text{token}}^3(x_q, x_{\text{inst}}, \tilde{x}_{\text{inst}}, y^{0:l}) = f(x_{\text{inst}}, x_q, y^{0:l}) \cdot f(\tilde{x}_{\text{inst}}, x_q, y^{0:l})^\top,$$

a dot-product similarity that captures the sensitivity of the model’s next-token prediction to perturbations in the system instruction. Compared with cosine similarity, the dot product more strongly emphasizes changes in distribution concentration, providing a sharper confidence signal.

Token-level contrastive confidence often exhibits a characteristic pattern: as shown in Figure 3, in a phrase, earlier tokens often have lower confidence than later ones, since the prefix increasingly constrains what can follow. To obtain a sequence-level confidence that reflects the point of instability, we aggregate token-level scores using a minimum operator:

$$C^3(x_q, x_{\text{inst}}, \tilde{x}_{\text{inst}}, y) = \min_{i,j \in \{0, \dots, l\}} \left(\text{diag}(f(x_{\text{inst}}, x_q, y^{0:l})[i:j] \cdot f(\tilde{x}_{\text{inst}}, x_q, y^{0:l})[i:j]^\top) \right). \quad (1)$$

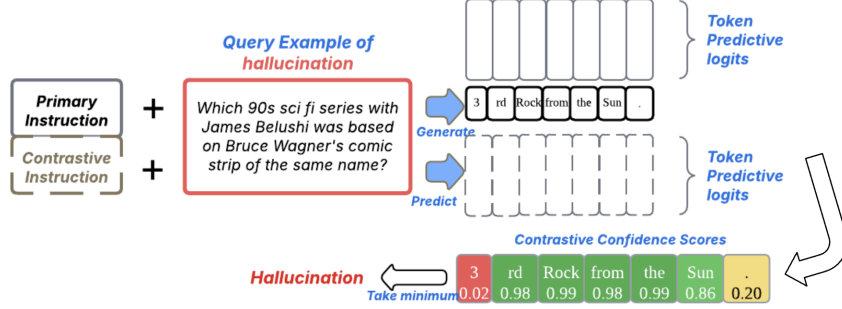


Fig. 2: Overview C^3 . Given a query, C^3 compares output logits generated from a primary instruction and a contrastive instruction. Confidence scores are computed at each token by contrasting the predictive logits.

Here, each logits matrix is of size $l \times v$ (response length $\times$ vocabulary size), and computing the contrastive scores requires only one additional forward pass beyond the initial generation.

Crucially, the LLM doesn't generate a new response with contrastive instruction. We feed the response y to the model and run with teacher forcing to collect the logits. Thus, the difference of logits is attributable to the system-instruction perturbation rather than diverging responses.

Algorithm 1 Inference-Phase C^3 Confidence Computation

Inputs: Instruction prompt x , Contrastive instruction prompt $\tilde{x}$, Slice of response S , with start index S_s and end index S_e , The i th question $Q_i \in Q = \{Q_1, \dots, Q_m\}$, Language model f

Output: Confidence scores C^3_i

- 1: **(A) Primary Response Generation**
 - 2: $y_i \leftarrow f(x, Q_i)$
 - 3: **(B) Contrastive Confidence Computation**
 - 4: **return** $C^3_i \leftarrow \min(\text{diag}(f(x, Q_i, y_i)[S_s:S_e] \cdot f(\tilde{x}, Q_i, y_i)[S_s:S_e]))$
-

Time complexity analysis of Inference-Phase C^3 computation. Let $C_f(L+T)$ denote the time required for a single autoregressive forward pass of the language model f with context length L and response length T . Algorithm 1 incurs this cost twice: once for generating the primary response and once for producing the contrastive logits under the perturbed system instruction. Both steps require assembling a prompt and running the LLM, giving costs $T_A = O(C_f(L+T))$ and $T_B^{\text{LM}} \approx O(C_f(L+T))$, respectively. Because the contrastive prompt is only marginally longer than the primary prompt, the two forward passes are of the same asymptotic order.

Beyond these LLM evaluations, the additional computation is minimal. Computing the contrastive confidence involves multiplying two matrices of size $L_{\text{slice}} \times d$ and applying a token-wise minimum, yielding $O(L_{\text{slice}}d)$ work per query or

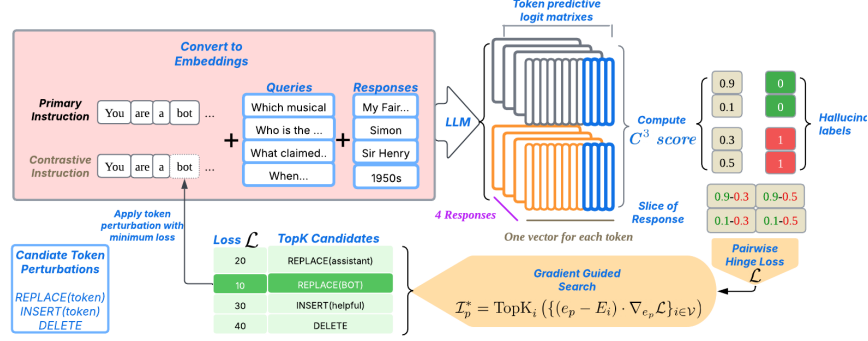


Fig. 3: The gradient-guided perturbation search framework C^3 . At each step, the method applies candidate token edits to the instruction in system message to form a contrastive instruction $\tilde{x}_{\text{inst}}$. The modified system instruction is concatenated with a batch of user queries and fed into the LLM to obtain token-level predictive logits. From these logits, C^3 scores are computed for each response. The pairwise hinge loss—measuring how well the current perturbation separates hallucinated from non-hallucinated samples—is then evaluated, and gradient-guided search selects the top- k candidate token edits based on Eq. 3. The perturbation with the lowest loss is applied to update the system instruction.

$O(m L_{\text{slice}} d)$ across all m queries. Since L_{slice} and d are small relative to the cost of LLM forward passes, this overhead is negligible. Thus, the dominant runtime of Algorithm 1 part (B) is effectively *one forward passes* over the generated response of part (A), giving C^3 a significant efficiency advantage over sampling-based baselines that require 5–10 generations per query.

3.2 Gradient-Based Contrastive Perturbation Search

We now describe how to automatically learn the contrastive system instruction $\tilde{x}_{\text{inst}}$ that maximizes the separation in the conditional contrastive confidence $C^3(Q, x_{\text{inst}}, \tilde{x}_{\text{inst}}, y)$ between hallucinated and non-hallucinated responses. Unlike standard prompt engineering—which relies on manual edits—our goal is to learn small, targeted perturbations to the system instruction that reliably expose model uncertainty while leaving the user query untouched.

Our method follows the two-stage structure used in AutoPrompt [16] and Universal Adversarial Triggers [19]: (1) a *gradient-based candidate discovery* stage to identify promising token substitutions, and (2) a *discrete search* stage that evaluates these candidates and selects the perturbation that minimizes the training loss. To allow flexible editing beyond simple substitutions, we additionally support deletion and unbounded insertion of tokens into the system instruction.

Pairwise Hinge Loss Objective. To learn $\tilde{x}_{\text{inst}}$, we optimize a pairwise hinge loss that encourages non-hallucinated responses to obtain higher contrastive confidence than hallucinated ones. Let $P = \{p_i\}$ denote hallucinated (positive) samples and $N = \{n_j\}$ non-hallucinated (negative) samples. For each pair (p_i, n_j) , we enforce a margin between the scores of hallucinated and non-hallucinated

responses, which specifies how much separation C^3 must achieve before it stops being penalized. Since $C^3 \in [0, 1]$, the effective range of *margin* is $(0, 1]$. A small positive margin means the loss stops penalizing once the non-hallucinated sample’s confidence exceeds the hallucinated sample’s by even a slight amount. In contrast, when *margin* = 1, the loss penalizes almost all score pairs, and is only 0 under the extreme case where the non-hallucinated response attains $C^3 = 1$ and the hallucinated response attains $C^3 = 0$:

$$\mathcal{L}(\text{conf}^{0:m}, h^{0:m}) = \frac{1}{|P||N|} \sum_{i=1}^{|P|} \sum_{j=1}^{|N|} \max(0, \text{margin} - (\text{conf}^{n_j} - \text{conf}^{p_i})), \quad (2)$$

where

$$\text{conf}^{p_i} = C^3(Q^{p_i}, x_{\text{inst}}, \tilde{x}_{\text{inst}}, y^{p_i}), \quad \text{conf}^{n_j} = C^3(Q^{n_j}, x_{\text{inst}}, \tilde{x}_{\text{inst}}, y^{n_j}).$$

Minimizing this loss yields contrastive instructions that amplify C^3 -based distinctions between the two classes.

Gradient-Guided Candidate Identification. Because $\tilde{x}_{\text{inst}}$ is composed of discrete tokens, direct gradient descent is not feasible. Following [19], we use gradients in embedding space to approximate how replacing a token would affect the hinge loss.

Let $E \in \mathbb{R}^{V \times d}$ be the LLM’s vocabulary embedding matrix, where V is vocabulary size and d embedding dimension. For a perturbable position p , the current system-instruction token is x_p with embedding $e_p = E_{x_p} \in \mathbb{R}^d$. We compute the gradient of the hinge loss with respect to e_p :

$$\nabla_{e_p} \mathcal{L} \in \mathbb{R}^d.$$

To estimate how substituting x_p with another token i changes the loss, we use a first-order Taylor approximation:

$$\mathcal{I}_p^* = \text{TopK}_i \left\{ (e_p - E_i) \cdot \nabla_{e_p} \mathcal{L} \mid i \in \mathcal{V} \right\}, \quad (3)$$

where each inner product $(e_p - E_i) \cdot \nabla_{e_p} \mathcal{L}$ approximates the directional change in the loss induced by replacing x_p with token i . The resulting index set $\mathcal{I}_p^*$ contains the top- k candidate substitutions predicted to most reduce the loss. These candidates are then evaluated in the discrete stage, where we select the perturbation that yields the lowest hinge loss and thus the most discriminative contrastive instruction. The time complexity of the perturbation search is $O(|P|T((k+2)C_{\text{conf}} + Vd))$, detailed in the supplementary document.

4 Empirical Evaluation

4.1 Settings

LLM Backbones. We evaluate our method using several LLMs: instruction-tuned Llama 3.2 1B, 3B, Llama 3.1 8B, Qwen2 1.5B and Gemma 2 2B¹.

¹ Note that Gemma does not provide system prompt tags, so we insert the system instruction at the beginning of the user prompt.

Baselines. We benchmark our method against sampling-based methods: Semantic Entropy [7], SelfCheckGPT [10], Degree [8], Kernel Language Entropy [11] and SPUQ [2]; the self-evaluation method P-True [4]; the traditional mean token entropy method and log-likelihood [1].

Datasets. All methods are compared on TriviaQA, a free-form question-answering dataset, and SST-2, a sentiment classification dataset. Following previous research, correctness of responses is assessed using Rouge-L scores: a response is deemed correct if its Rouge-L score with the reference answer exceeds a threshold 0.3 following previous works [13,6]. Prior work indicates that baseline performance is highly sensitive to the chosen Rouge-L threshold. To exclude this factor, we incorporate the SST-2 sentiment classification task into our experiments, as sentiment classification accuracy is not affected by variations in Rouge-L thresholds. To ensure size consistency among datasets, we subsample their validation sets to 1k examples following [7]. As our C^3 method has no randomness during inference, we select the first 1k samples for reproducibility. Gradient-guided search utilizes 1k training samples as well.

Hyperparameter and Design Choices. The C^3 scoring function has no tunable hyperparameters at inference time; we evaluate contrast across the full generated response. The gradient-guided search stage includes several design choices: hinge-loss margin, modifiable positions, number of search steps, and number of candidate tokens per step. We use a default hinge margin of 1, all positions in the first sentence of the instruction as modifiable positions, 3 search steps and 5 candidate tokens per step, exploring 15 tokens in total out of a vocabulary of $\sim 100k$, relying heavily on gradient signals for efficiency.

Evaluation Metrics. Following standard conventions in LLM UQ performance evaluation, we report the area under the receiver operator characteristic curve (AUROC) and the area under the precision-recall curve (AUPR).

Other information about the experiments, such as system instructions of the backbones and pseudocode of the training-phase algorithm, hyperparameter selection of the baselines, as well as the code and datasets, are provided in the supplementary document².

4.2 Results

Performance Comparison. Performance Comparison. Sampling-based methods such as Semantic Entropy, SelfCheckGPT, Degree and KLE provide strong baseline performance across both datasets. However, C^3 achieves the highest AUROC and AUPR in 8 of 10 and 9 of 10 comparisons, respectively, showing consistent advantages across multiple backbones and tasks.

² <https://github.com/Tano-Y/Conditional-Contrastive-Confidence-Based-Uncertainty-Quantification-for-LLMs>

Table 1: AUROC and AUPR scores for different uncertainty quantification methods across LLM backbones and tasks: (a) TriviaQA (question answering) and (b) SST-2 (sentiment classification).

(a) TriviaQA

AUROC/AUPR	LLama 3.2 1b	LLama 3.2 3b	LLama 3.1 8b	Qwen2 1.5b	Gemma 2b	Average
Accuracy	40.3	68.2	68.4	37.7	58.3	54.6
SE-5	0.719/0.752	0.815/0.621	0.765/0.545	0.737/0.785	0.764/0.666	0.760/0.674
SE-10	0.759/0.781	0.839/0.697	0.800/0.645	0.756/0.797	0.766/0.679	0.784/0.720
SelfCKGPT-5	0.774/0.789	0.818/0.682	0.796/0.658	0.763/0.806	0.770/0.680	0.784/0.723
SelfCKGPT-10	0.785/0.803	0.835/0.707	0.822/0.675	0.784/0.824	0.786/0.704	0.802/0.742
Degree-5	0.746/0.771	0.792/0.616	0.755/0.601	0.757/0.804	0.766/0.666	0.763/0.691
Degree-10	0.761/0.787	0.814/0.666	0.782/0.623	0.782/0.818	0.790/0.694	0.786/0.718
KLE Heat-10	0.753/0.761	0.823/0.661	0.808/0.647	0.757/0.784	0.787/0.693	0.786/0.709
LN-Entropy-5	0.651/0.707	0.773/0.559	0.756/0.558	0.666/0.732	0.739/0.655	0.717/0.642
LN-Entropy-10	0.638/0.704	0.791/0.600	0.779/0.583	0.654/0.741	0.734/0.647	0.719/0.655
MTE	0.682/0.743	0.773/0.578	0.690/0.521	0.682/0.754	0.767/0.692	0.719/0.658
Log-likelihood	0.732/0.784	0.808/0.628	0.734/0.576	0.690/0.774	0.757/0.681	0.744/0.689
P(True)	0.407/0.525	0.533/0.360	0.549/0.372	0.523/0.671	0.352/0.342	0.473/0.454
SPUQ Tokens	0.716/0.739	0.803/0.590	0.776/0.561	0.737/0.793	0.773/0.655	0.761/0.668
SPUQ Para.	0.643/0.678	0.728/0.478	0.622/0.429	0.641/0.708	0.638/0.543	0.654/0.567
C ³	0.801/0.831	0.862/0.755	0.847/0.692	0.774/0.827	0.794/0.735	0.816/0.768

(b) SST-2

AUROC/AUPR	LLama 3.2 1b	LLama 3.2 3b	LLama 3.1 8b	Qwen2 1.5b	Gemma 2b	Average
Accuracy	68.3	74.4	92.7	91.7	92.4	83.9
SE-5	0.454/0.317	0.495/0.259	0.551/0.164	0.536/0.149	0.568/0.129	0.521/0.204
SE-10	0.558/0.371	0.573/0.295	0.640/0.189	0.584/0.134	0.587/0.145	0.588/0.227
SelfCKGPT-5	0.613/0.393	0.522/0.312	0.543/0.202	0.748/0.225	0.598/0.197	0.605/0.266
SelfCKGPT-10	0.632/0.391	0.576/0.334	0.603/0.250	0.793/0.283	0.680/0.293	0.657/0.310
Degree-5	0.640/0.408	0.526/0.314	0.553/0.215	0.749/0.205	0.599/0.233	0.614/0.275
Degree-10	0.643/0.397	0.580/0.346	0.603/0.262	0.785/0.305	0.680/0.286	0.658/0.319
KLE Heat-10	0.514/0.331	0.515/0.267	0.593/0.131	0.678/0.166	0.566/0.124	0.573/0.204
LN-Entropy-5	0.658/0.421	0.753/0.420	0.704/0.232	0.779/0.189	0.856/0.363	0.750/0.325
LN-Entropy-10	0.639/0.393	0.761/0.434	0.722/0.305	0.775/0.258	0.858/0.367	0.751/0.351
MTE	0.709/0.445	0.774/0.461	0.824/0.320	0.865/0.354	0.865/0.405	0.807/0.397
Log-likelihood	0.706/0.420	0.769/0.448	0.643/0.245	0.871/0.394	0.863/0.404	0.770/0.382
P(True)	0.551/0.345	0.774/0.622	0.839/0.286	0.707/0.145	0.350/0.054	0.644/0.290
SPUQ Tokens	0.548/0.198	0.629/0.283	0.723/0.271	0.634/0.191	0.688/0.225	0.644/0.233
SPUQ Para.	0.746/0.490	0.771/0.502	0.806/0.221	0.738/0.182	0.479/0.073	0.708/0.294
C ³	0.846/0.767	0.925/0.845	0.912/0.438	0.862/0.306	0.881/0.519	0.885/0.575

On TriviaQA, C³ attains the best average AUROC/AUPR of 0.816 / 0.768 on the 5 backbones, surpassing the best baseline SelfCheckGPT-10 by +0.014 and +0.026, respectively. Comparing with the 5 alternative version, which has less computational cost than 10 alternative ones, it is +0.032 and +0.045. This improvement indicates that C³ provides more reliable uncertainty discrimination without relying on extensive sampling. On SST-2, C³ again delivers superior performance. It achieves 0.885 / 0.575 on average, improving over the best baseline MTE by +0.078 in AUROC and +0.178 in AUPR.

Overall, C³ demonstrates a balanced trade-off between accuracy and computational efficiency, achieving competitive or superior uncertainty quantification without resorting to costly sampling.

Generalization across LLM Backbones. Blackbox Applicability. Figure 4 examines the generalization performance of the searched perturbations when applied to unseen models (off-diagonal entries). Among the six evaluated models, four share similar instruction-tuning objectives, allowing meaningful transfer of perturbations across them. We compare our method with the baseline that achieves best average performance. On SST-2, the highest baseline is MTE. Our results demonstrate that in 10 out of 12 cases C^3 shows comparable or higher AUROC and 9 out of 12 show comparable or higher AUPR when applying perturbations to unseen target models.

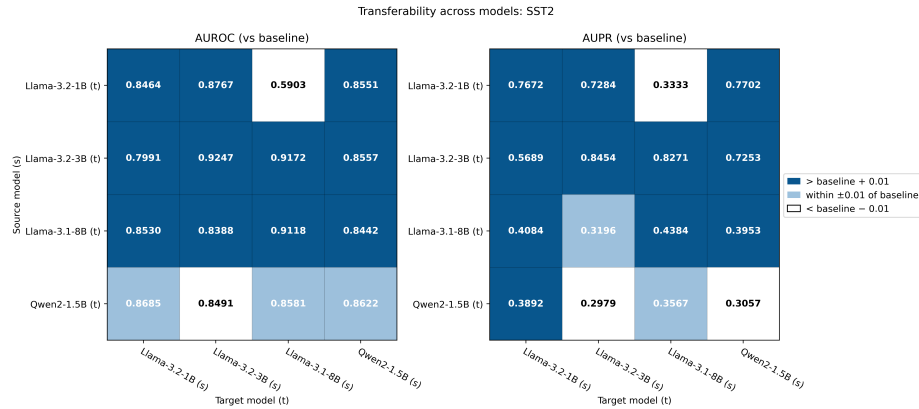


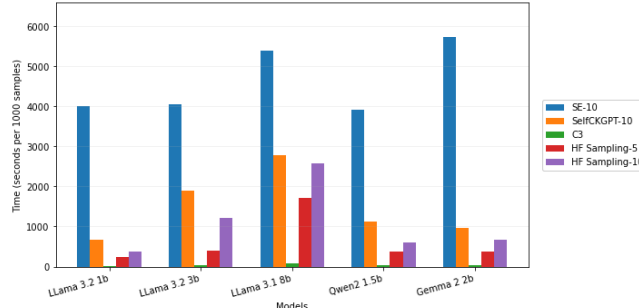
Fig. 4: Based on the intuitions derived from searched perturbations of source models. This chart shows their generalization ability to unseen models (target models) as demonstrated in SST2. Similar patterns are observed on TriviaQA.

Ablation Study: Our method explicitly measures contrast among model outputs and therefore naturally adopts max aggregation. Because prior work typically aggregates by the mean, we ran an ablation to test whether our gains arise solely from the aggregator. Replacing mean with max in the baselines improves their performance, but they still underperform our method, highlighting the necessity of the explicit contrastive signal. Across model sizes (1B/3B/8B), swapping the mean-aggregated baseline (MTE) for max aggregation (“Max TE”) consistently lifts both metrics—AUROC by +5.8–10.6 points and AUPR by +6.0–12.9 points on the three backbones. Yet our method (C^3) remains superior for every backbone, adding a further +3.0–5.0 AUROC points and +2.8–9.0 AUPR points over Max TE. This monotonic ordering (MTE < Max TE < C^3) across small, mid, and larger models indicates that while max aggregation helps, it does not close the gap—explicit contrastive information is the critical source of the gains rather than the choice of aggregator.

Computational Efficiency: We selected the four most representative methods to measure inference runtime and compare with. HF sampling-5 and 10 are the running times of sampling 5 and 10 responses with standard huggingface model calls. They represent the amount of times that sampling methods must all have that is not reducible. They show the lower bound of running time for the family

Table 2: Ablation results on AUROC/AUPR under different configurations.

AUROC/AUPR	MTE	Max TE	C ³
LLama 3.2 1b	0.6822 / 0.7433	0.7570 / 0.8029	0.8008 / 0.8313
LLama 3.2 3b	0.7729 / 0.5780	0.8313 / 0.6645	0.8616 / 0.7545
LLama 3.1 8b	0.6903 / 0.5212	0.7963 / 0.6500	0.8467 / 0.6923

Fig. 5: Inference running time comparison of Semantic Entropy and SelfCheckGPT with 10 samples, C³, sampling 5 and 10 responses with Hugging Face API.

of sampling-based methods. SelfcheckGPT represents the methods that use NLI to measure similarities of one response to the rest. Semantic Entropy represents the methods that use NLI to measure all pair-wise similarities among responses.

Figure 5 reports the empirical runtime measurements of different uncertainty estimation methods. Empirical benchmarks on 1k examples per dataset show that C³ runs approximately 36.31x faster than SelfCheckGPT and 113.08x faster than Semantic Entropy with 10 samples. 15.24x faster than sampling 5 responses and 26.60x faster than sampling 10 responses with huggingface.

5 Conclusion

In this work, we introduced Conditional Contrastive Confidence (C³), a novel uncertainty quantification framework designed to efficiently and effectively identify uncertainty in large language model outputs. C³ leverages the phenomenon that when LLMs are confident, they exhibit stable predictions under minor instructional perturbations. Our experimental results demonstrate that C³ matches or surpasses existing sampling-based baselines across diverse models and benchmark datasets, significantly reducing inference time. Future research could explore extending C³ to long-form generation tasks, examining its potential for pinpointing uncertainty in specific sections of model-generated content, thus aiding in targeted response revisions. Additionally, further refinement of the search algorithm may yield even more interpretable and broadly applicable contrastive instructions, enhancing both the practicality and reliability of uncertainty quantification in critical LLM applications.

Acknowledgment. This work was supported by the National Institute of Standards and Technology (NIST) under Grant No. 14147259.

References

1. Fadeeva, E., et al.: LM-polygraph: Uncertainty estimation for language models. In: EMNLP (2023)
2. Gao, X., et al.: SPUQ: Perturbation-based uncertainty quantification for large language models. In: EACL (2024)
3. Huang, X., et al.: Uncertainty in language models: Assessment through rank-calibration. In: EMNLP (2024)
4. Kadavath, S., et al.: Language models (mostly) know what they know. arXiv preprint arXiv:2207.05221 (2022)
5. Kim, S.S., et al.: "i'm not sure, but...": Examining the impact of large language models' uncertainty expression on user reliance and trust. In: ACM FAccT. pp. 822–835 (2024)
6. Kossen, J., et al.: Semantic entropy probes: Robust and cheap hallucination detection in llms. arXiv preprint arXiv:2406.15927 (2024)
7. Kuhn, L., et al.: Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. arXiv preprint arXiv:2302.09664 (2023)
8. Lin, Z., et al.: Generating with confidence: Uncertainty quantification for black-box large language models. TMLR (2024)
9. Mahaut, M., et al.: Factual confidence of LLMs: on reliability and robustness of current estimators. In: ACL (2024)
10. Manakul, P., et al.: Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. arXiv preprint arXiv:2303.08896 (2023)
11. Nikitin, A., et al.: Kernel language entropy: Fine-grained uncertainty quantification for llms from semantic similarities. NeurIPS **37**, 8901–8929 (2024)
12. Press, O., et al.: The entropy enigma: Success and failure of entropy minimization. arXiv preprint arXiv:2405.05012 (2024)
13. Qiu, X., Mikkilainen, R.: Semantic density: Uncertainty quantification for large language models through confidence measurement in semantic space. NeurIPS **37**, 134507–134533 (2024)
14. Sclar, M., et al.: Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. arXiv preprint arXiv:2310.11324 (2023)
15. Shi, W., et al.: Trusting your evidence: Hallucinate less with context-aware decoding. In: NAACL. pp. 783–791 (2024)
16. Shin, T., et al.: AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In: EMNLP (2020)
17. Siska, C., et al.: Examining the robustness of llm evaluation to the distributional assumptions of benchmarks. In: ACL. pp. 10406–10421 (2024)
18. Vaswani, A.: Attention is all you need. NeuralIPS (2017)
19. Wallace, E., et al.: Universal adversarial triggers for attacking and analyzing NLP. In: EMNLP-IJCNLP (2019)
20. Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171 (2022)
21. Xiong, M., et al.: Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In: ICLR (2024)
22. Yang, A., et al.: Just rephrase it! uncertainty estimation in closed-source language models via multiple rephrased queries. arXiv preprint arXiv:2405.13907 (2024)
23. Zhang, C., et al.: Prefer: Prompt ensemble learning via feedback-reflect-refine. AAAI **38**(17), 19525–19532 (Mar 2024)